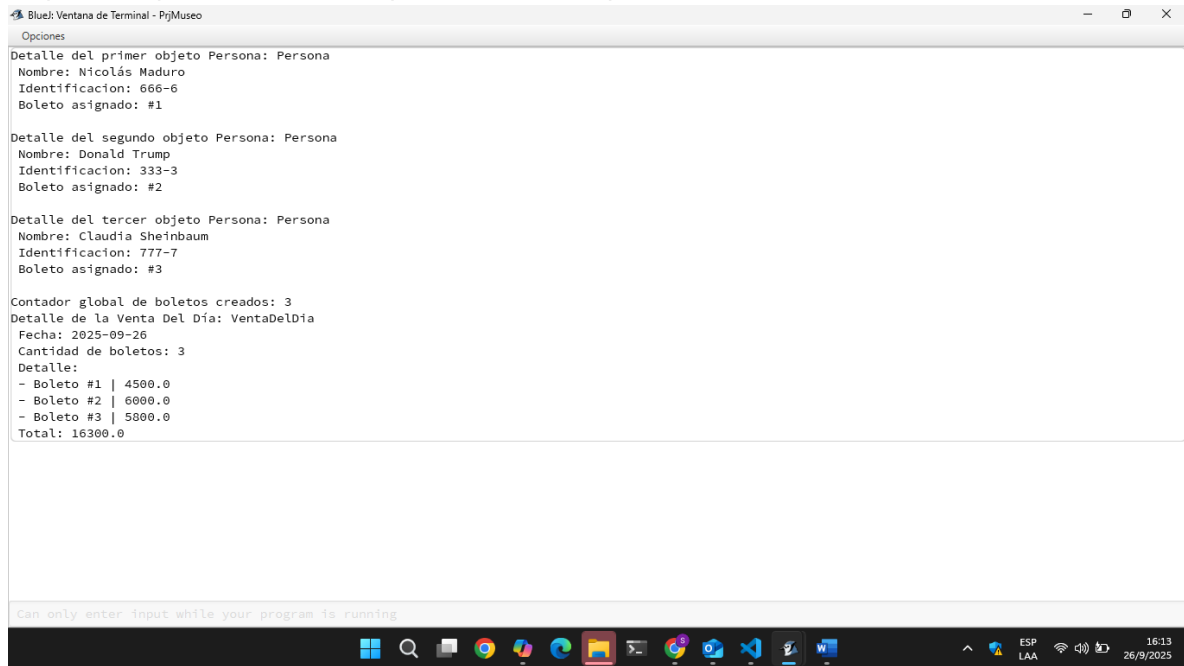


índice E)

Captura de pantalla de la compilación de PrincipalMuseo



```
Blue: Ventana de Terminal - PjMuseo
Opciones
Detalle del primer objeto Persona: Persona
Nombre: Nicolás Maduro
Identificación: 666-6
Boleto asignado: #1

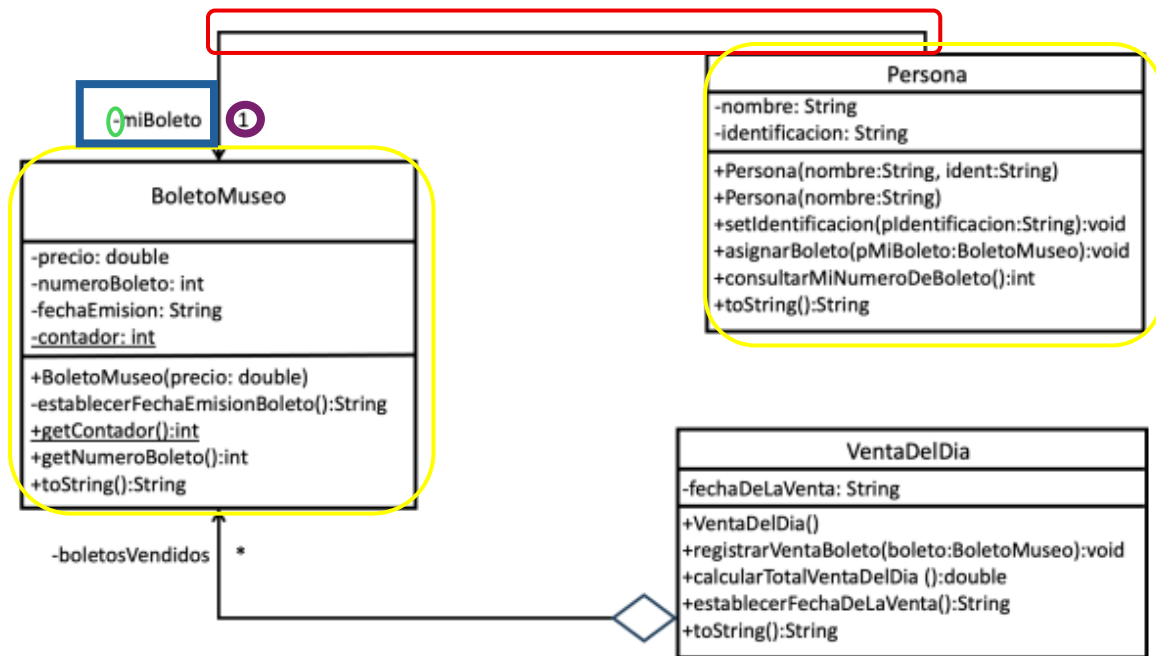
Detalle del segundo objeto Persona: Persona
Nombre: Donald Trump
Identificación: 333-3
Boleto asignado: #2

Detalle del tercer objeto Persona: Persona
Nombre: Claudia Sheinbaum
Identificación: 777-7
Boleto asignado: #3

Contador global de boletos creados: 3
Detalle de la Venta Del Día: VentaDelDia
Fecha: 2025-09-26
Cantidad de boletos: 3
Detalle:
- Boleto #1 | 4500.0
- Boleto #2 | 6000.0
- Boleto #3 | 5000.0
Total: 16300.0

Can only enter input while your program is running
```

índice H):



(1) Representación UML - **Amarillo**

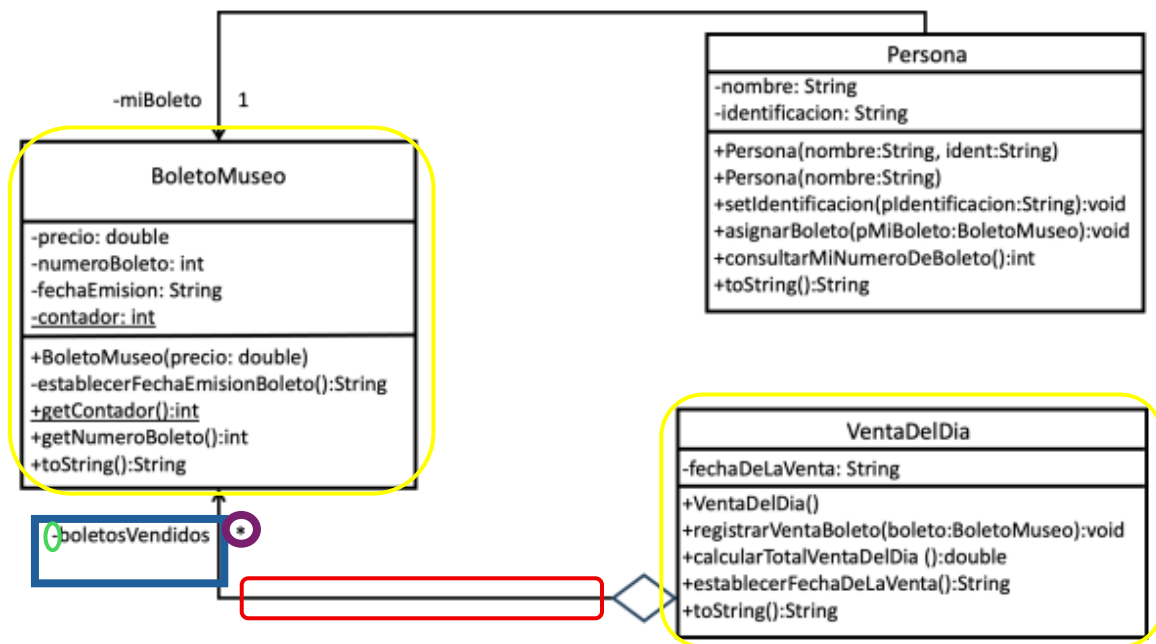
(2) Navegabilidad – **Rojo**

(3) Rol - **Azul**

(4) Modificador del rol - **Verde**

(5) Cardinalidad - **Morado**

índice I):



(1) Representación UML - **Amarillo**

(2) Navegabilidad - **Rojo**

(3) Rol - **Azul**

(4) Modificador del rol - **Verde**

(5) Cardinalidad - **Morado**

Índice J)

a. Asociación entre clase A y clase B

No, la estructura de la clase B no se ve impactada directamente.

En una relación de asociación, una clase puede establecer una comunicación unidireccional o bidireccional con otra, pero esto no implica una dependencia estructural mutua. Por ejemplo, si la clase A tiene una referencia a B, A puede interactuar con B sin que B necesite modificar su estructura ni declarar atributos relacionados con A. Esto permite mantener bajo acoplamiento y autonomía entre clases.

b. Agregación entre clase P y clase Q

No, la estructura de la clase Q no se ve afectada por la agregación con P.

La agregación representa una relación “todo-parte” donde P agrupa instancias de Q, pero sin poseerlas completamente. Las instancias de Q pueden existir independientemente de P, incluso si P se elimina. Además, P no modifica la lógica interna de Q ni altera su estado directamente, lo que preserva la independencia estructural de la clase parte.

c. Asociación bidireccional entre clase A y clase B

Sí, la estructura de ambas clases se ve impactada en cuanto a la declaración de referencias mutuas, pero no en su lógica interna.

Cuando A y B están asociadas bidireccionalmente, cada una debe declarar un atributo que referencia a la otra. Esto modifica su estructura al nivel de atributos, pero ambas siguen siendo clases independientes en cuanto a comportamiento. La relación permite comunicación mutua sin que una clase controle o altere la lógica de la otra.

d. Comunicación entre objetos Z y W sin vínculo estructural

Sí, un objeto de tipo Z puede enviar mensajes a un objeto de tipo W sin que exista una relación estructural entre sus clases.

Esto es posible gracias a la flexibilidad del paradigma orientado a objetos, donde lo importante es la referencia entre instancias en tiempo de ejecución. Z puede recibir una instancia de W como parámetro, retorno de método o variable local, y así invocar sus métodos sin que exista una asociación o agregación formal entre sus clases.

e. ¿Un diagrama con los cinco elementos garantiza el vínculo?

No necesariamente. Aunque el diagrama incluya los cinco elementos (representación UML, navegabilidad, rol, modificador del rol y cardinalidad), el vínculo solo se establece completamente si el código respeta fielmente ese diseño.

Errores como declarar un tipo incorrecto, usar un modificador de acceso distinto, omitir el atributo o no respetar la cardinalidad pueden romper el vínculo. El diagrama es una guía estructural, pero debe ser respaldado por una implementación coherente en el código fuente.

Indice L)

Reflexión de Aprendizaje

Durante el desarrollo de esta actividad, fortalecí significativamente mi comprensión sobre las clases en Java, sus estructuras internas y los distintos tipos de vínculos que pueden establecer entre sí, como la asociación, agregación y composición. Aprendí a identificar y representar correctamente estos vínculos en diagramas UML, reconociendo elementos clave como la navegabilidad, el rol, el modificador del rol y la cardinalidad, lo que me permitió interpretar y construir modelos más precisos y profesionales.

Además, profundicé en el uso de Javadoc, comprendiendo la función de etiquetas como, `@param`, `@return`, `@autor`, `@version`, y `,`, lo cual me ayudó a documentar mis clases de forma clara, estandarizada y útil para otros desarrolladores.

También mejoré mi manejo de BlueJ, tanto en la creación de diagramas como en la implementación de código, lo que me permitió visualizar mejor las relaciones entre clases y validar su comportamiento en tiempo de ejecución. En conjunto, estos aprendizajes me permitieron comprender con mayor profundidad el diseño orientado a objetos, y aplicar buenas prácticas tanto en modelado como en codificación.